

Projeto 1: Listas Lineares / ArrayList / Java IO

Faremos a implementação (em **Java**) de uma versão simplificada de um carrinho de compras de uma determinada loja:

	item	Preço	Quantidade	Subtotal
1	Tesoura Molin 17Cm Blue Titanium	R\$ 25,67	1	R\$ 25,67
2	Conjunto Geométrico Super Molin C/ 4	R\$ 17,39	2	R\$ 34,78
3	Caderno Universitário Espiral Capa Dura 10X1 200 Folhas	R\$ 32,11	1	R\$ 32,11
	Subtotal			R\$ 100,60
	Descontos			R\$ -8,04
	TOTAL			R\$ 92,56

Figura 1: Carrinho de compras.

Base de dados

Os dados dos produtos estão em um arquivo chamado `produtos.txt` e cada linha do arquivo representa um produto do estoque, com as informações separadas por ponto-e-vírgula:

`Código;Descrição do produto;Preço unitário`

Exemplo:

```
12345;Tesoura Molin 17Cm Blue Titanium;25.67
67890;Conjunto Geométrico Super Molin C/4;17.39
56478;Caderno Universitário Espiral 200 Fls;32.11
```

Requisitos do sistema

Seu programa deverá:

1. Transferir todo o conteúdo do arquivo `produtos.txt` para um `ArrayList` de objetos do tipo `Produto`, que tem a seguinte estrutura:

Produto
- <code>codigo: int</code>
- <code>descricao: String</code>
- <code>preco: double</code>
+ <code>Produto()</code>

Métodos:

- `Produto()`: O método construtor deve receber todos os parâmetros (`codigo`, `descricao`, `preco`) e armazenar nos respectivos atributos da classe.
 - Incluir os métodos *getters* e *setters* que você ache necessário para o projeto.
2. Permitir que o usuário informe o código do produto para fazer uma busca no `ArrayList`; se encontrar o código, o usuário deve informar a quantidade a ser comprada do produto. Com essas informações, criar um objeto do tipo `ItemCompra` para adicioná-lo posteriormente ao `Carrinho`. Essas informações (`produto`, `quantidade`) compõem um item de compra que deve ter a seguinte estrutura:

ItemCompra
- <code>produto: Produto</code>
- <code>quantidade: int</code>
+ <code>ItemCompra()</code>
+ <code>mostrar(): void</code>
+ <code>obterSubtotal(): double</code>

Métodos:

- `ItemCompra()`: O método construtor deve receber todos os parâmetros e armazenar nos respectivos atributos da classe.
 - `mostrar()`: Exibir em uma única linha, a descrição, o preço (obtidos via objeto `produto`), a quantidade e o subtotal (obtido com o método `obterSubtotal()`) do item de compra.
 - `obterSubtotal()`: Calcular e retornar o subtotal do item de compra (quantidade * preço unitário).
3. Criar a classe `Carrinho`, com a seguinte estrutura:

Carrinho
- listaItens: ArrayList<ItemCompra> - porcentagemDesconto: double
+ Carrinho() + mostrar(): void + obterValorDesconto(): double + obterSubtotal(): double + obterTotal(): double

Métodos:

- **Carrinho()**: O método construtor deve receber um **ArrayList** preenchido com itens de compra e a porcentagem de desconto como parâmetro e armazenar nos respectivos atributos da classe.
- **mostrar()**: Exibir as informações do carrinho conforme a **Figura 1**, seguindo os requisitos abaixo:
 - i. O conteúdo do **ArrayList** (itens comprados) deve ser exibido de forma numerada e usar o método **mostrar()** da classe **ItemCompra**.
 - ii. O subtotal da compra deve usar o método **obterSubtotal()**.
 - iii. O desconto deve usar o método **obterValorDesconto()**.
 - iv. O total da compra deve usar o método **obterTotal()**.
- **obterValorDesconto()**: Calcular e retornar o desconto para o carrinho, a partir do subtotal da compra (chamar o método **obterSubtotal()**) e do percentual de desconto armazenado na classe **Carrinho**.
- **obterSubtotal()**: Calcular e retornar o subtotal da compra, a partir da somatória de todos os subtotais armazenados no **ArrayList** **listaItens**.
- **obterTotal()**: Calcular e retornar o total final da compra, usando os métodos **obterSubtotal()** e **obterValorDesconto()**.

Para a classe de teste que contém o método **main()**:

- Carregue o conteúdo do arquivo **produtos.txt** no **ArrayList** de **Produto**.
- Em um *loop*, peça ao usuário para digitar o código de um **Produto**. Caso o código seja válido, peça ao usuário para digitar a quantidade que será comprada. Caso a quantidade seja válida, a compra deve ser inserida no **ArrayList** de **ItemCompra**. Caso o usuário digite **0** como código do produto, o programa deve entender que o usuário não quer mais incluir itens na lista de compras e o *loop* deve ser encerrado.
- Com o **ArrayList** de **ItemCompra** contendo as informações digitadas pelo usuário, crie um **Carrinho** de compras com essa lista e um percentual de desconto de 10%.
- Por fim, exiba o **Carrinho** de compras com todas as informações, conforme o modelo da **Figura 1**.

Critérios de avaliação

O projeto será avaliado de acordo com os critérios a seguir (Tabela 1).

Item de avaliação	Pontuação máxima	Parcial	Ausente
1. Estrutura das classes do projeto	1,00	0,50	0,00
Classe Produto	(0,25) Atende aos critérios	(0,125) Parcial ou com falhas	Ausente
Classe ItemCompra	(0,25) Atende aos critérios	(0,125) Parcial ou com falhas	Ausente
Classe Carrinho	(0,25) Atende aos critérios	(0,125) Parcial ou com falhas	Ausente
Classe de teste (contendo o método main())	(0,25) Atende aos critérios	(0,125) Parcial ou com falhas	Ausente
2. Conteúdo do arquivo para ArrayList	1,50	0,75	0,00
Carregar o conteúdo do arquivo produtos.txt no ArrayList de Produto .	Atende aos critérios	Parcial ou com falhas	Ausente
3. Leitura de código e quantidade informado pelo usuário	2,00	1,00	0,00
<i>Loop</i> com a leitura de código e quantidade de um produto.	(0,50) Atende aos critérios	(0,25) Parcial ou com falhas	Ausente
Encerramento do <i>loop</i> quando o código digitado for 0 .	(0,50) Atende aos critérios	(0,25) Parcial ou com falhas	Ausente
Validação da entrada do usuário.	(1,00) Atende aos critérios	(0,50) Parcial ou com falhas	Ausente
4. Criação do carrinho de compras	1,00	0,50	0,00
Criação correta do carrinho de compras, a partir da lista de itens de compra gerada com as informações digitadas pelo usuário.	Atende aos critérios	Parcial ou com falhas	Ausente
5. Exibição do carrinho de compras	4,50	2,25	0,00
Exibição do carrinho de compras (a partir do objeto Carrinho).	(1,00) Atende aos critérios	(0,50) Parcial ou com falhas	Ausente
Exibição dos itens do carrinho (a partir de objetos ItemCompra).	(2,00) Atende aos critérios	(1,00) Parcial ou com falhas	Ausente
Exibição do subtotal da compra.	(0,50) Atende aos critérios	(0,25) Parcial ou com falhas	Ausente
Exibição do valor do desconto.	(0,50)	(0,25)	Ausente

	Atende aos critérios	Parcial ou com falhas	
Exibição do total da compra.	(0,50) Atende aos critérios	(0,25) Parcial ou com falhas	Ausente

Tabela 1 - Critérios de avaliação do projeto.

A Tabela 2 a seguir contém critérios de avaliação que podem reduzir a nota final da atividade.

Item indesejável	Penalidade
Projeto é cópia de projeto feito por outro grupo.	Projetos envolvidos são zerados
Projeto não usa a linguagem Java.	Projeto é zerado
Projeto não foi realizado em grupo de 2 a 4 pessoas.	Projeto é zerado
Projeto não compila ou possui erros graves (ex. travamento).	Projeto é zerado
Arquivos enviados em formato incorreto.	-1,0 ponto
Não há identificação do grupo no código-fonte.	-1,0 ponto
Arquivos e/ou pastas intermediárias que são criados no processo de compilação foram enviados junto com o código-fonte.	-1,0 ponto

Tabela 2: Critérios de avaliação que reduzem a nota final.

Entrega

- O projeto deve ser desenvolvido **em grupo (mínimo 2 pessoas, máximo 4 pessoas)**.
- Inclua, como comentário no início do arquivo `.java`, a matrícula e nome completo de cada pessoa integrante do grupo.
- Compacte todos os arquivos `.java` no formato `zip` e envie o arquivo compactado via link do Moodle (apenas uma pessoa do grupo deverá enviar o trabalho).
- Caso o projeto não seja entregue, o grupo automaticamente receberá nota zero referente ao projeto e à avaliação (ver quadro a seguir).

Data de entrega: até os primeiros 30 minutos da aula do dia 29/09/2026 (turma 3J12) ou 30/09/2026 (turma 3J11).

No dia 29/09/2026 (turma 3J12) ou 30/09/2026 (turma 3J11) será realizada uma avaliação (individual, de 50 minutos) referente ao projeto, com peso 50%. A avaliação será em formato de questões de múltipla escolha. Todas as questões serão baseadas em soluções genéricas do projeto (código).

Apêndice: Dicas

1) É possível separar uma `String` em substrings usando o método `split()`, que recebe como parâmetro uma `String` descrevendo uma expressão regular (*regex*). Analise os seguintes exemplos, que usam a mesma `String teste`.

```
String teste = "Hello_World_123\nTest_Bla bla bla";
```

Exemplo 1: `split("_")` separa string pelo símbolo *underscore* `_`.

```
String s1[] = teste.split("_");
for (int i = 0; i < s1.length; ++i) {
    System.out.println("s1[" + i + "] = " + s1[i]);
}
```

Saída do exemplo 1:

```
s1[0] = Hello
s1[1] = World
s1[2] = 123
Test
s1[3] = Bla bla bla
```

Exemplo 2: `split("\\r?\\n|\\r")` separa string pela quebra de linha `\r\n` ou `\n` ou `\r` (depende do sistema operacional/formato do arquivo texto).

```
String s2[] = teste.split("\\r?\\n|\\r");
for (int i = 0; i < s2.length; ++i) {
    System.out.println("s2[" + i + "] = " + s2[i]);
}
```

Saída do exemplo 2:

```
s2[0] = Hello_World_123
s2[1] = Test_Bla bla bla
```

Exemplo 3: `split("_|\\r?\\n|\\r")` separa string pelo *underscore* ou pela quebra de linha.

```
String s3[] = teste.split("_|\\r?\\n|\\r");
for (int i = 0; i < s3.length; ++i) {
    System.out.println("s3[" + i + "] = " + s3[i]);
}
```

Saída do exemplo 3:

```
s3[0] = Hello
s3[1] = World
s3[2] = 123
s3[3] = Test
s3[4] = Bla bla bla
```

2) A classe `Data` a seguir possui um método estático `loadTextFileToString()` que faz a leitura de um arquivo texto e retorna o conteúdo como uma única string.

```
public class Data {  
    /**  
     * Cria uma string com o conteúdo do arquivo texto passado no parâmetro  
     * {@code filename}.  
     *  
     * @param filename Nome do arquivo texto a ser lido pelo método.  
     * @return {@code String} contendo todo o conteúdo do arquivo texto.  
     * @throws FileNotFoundException Arquivo passado no parâmetro {@code  
     * filename} não existe (exceção do {@code FileInputStream}).  
     * @throws IOException Problema ao ler conteúdo do arquivo texto  
     * (exceção do {@code BufferedReader}).  
     */  
    public static String loadTextFileToString(String filename) throws  
    FileNotFoundException, IOException {  
        InputStream is = new FileInputStream(filename);  
        InputStreamReader isr = new InputStreamReader(is);  
        BufferedReader br = new BufferedReader(isr);  
  
        StringBuilder sb = new StringBuilder();  
        while (true) {  
            String line = br.readLine();  
            if (line == null)  
                break;  
            sb.append(line).append('\n');  
        }  
  
        is.close();  
  
        return sb.toString();  
    }  
}
```

O método estático `loadTextFileToString()` da classe `Data` está implementado de tal forma que, para ler um arquivo texto chamado `exemplo.txt` e salvar o conteúdo do arquivo em uma `String` `conteudo`, podemos escrever um código similar ao abaixo:

```
try {  
    String conteudo = Data.loadTextFileToString("exemplo.txt");  
} catch (IOException e) {  
    System.err.println("Problema ao abrir e ler o conteúdo do arquivo!");  
    e.printStackTrace();  
    System.exit(-1); // Se quiser encerrar programa com um código de erro.  
}
```